A PROJECT REPORT ON,

“Automated Testing of an E-Commerce Website Using Selenium WebDriver”

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

SUBMITTED BY

Vishwajeet Tulse	**B5533**
Pravin Misal	**B5474**
Rohan Gund	**B5438**



Sinhgad Institutes

DEPARTMENT OF COMPUTER ENGINEERING

NBN SINHGAD SCHOOL OF ENGINEERING, PUNE-41

SAVITRIBAI PHULE PUNE UNIVERSITY

2026 - 27



Sinhgad Institutes

CERTIFICATE

This is to certify that the project report entitles

"Automated Testing of an E-Commerce Website Using Selenium WebDriver"

Submitted by

Vishwajeet Tulse	**BE Roll No: B5533**
Pravin Misal	**BE Roll No: B5474**
Rohan Gund	**BE Roll No: B5438**

is a bonafide work carried out by them under the supervision of **Prof. S. P. Bendale** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University as a part of Laboratory Practice IV work syllabus (Final year Computer Engineering).

(Prof. S. P. Bendale)	**(Prof. S. P. Bendale)**
Subject In-charge	**Head of Department**

ACKNOWLEDGEMENT

It is indeed a great pleasure and moment of immense satisfaction for us to present a project report on "Automated Testing of an E-Commerce Website Using Selenium WebDriver" amongst a wide panorama that provided us inspiring guidance and encouragement, we take the opportunity to thank those who gave us the indebted assistance. We wish to extend our cordial gratitude with profound thanks to our internal guide Prof. S. P. Bendale for his everlasting guidance. It was his inspiration and encouragement which helped us in completing our project.

Our sincere thanks and deep gratitude to Head of Department, Prof. S. P. Bendale and other faculty members; but also to all those individuals involved both directly and indirectly for their help in all aspects of the project.

At last but not least we express our sincere thanks to our Institute's Principal Dr. Shivprasad Patil for providing us infrastructure and technical environment.

Vishwajeet Tulse
Pravin Misal
Rohan Gund

ABSTRACT

Every change to a web application risks breaking features that already worked. Running the same manual test cases again after every change is slow and error-prone, which is why stable and repeatable test cases are automated. This project builds an automated regression test suite for Swag Labs (`https://www.saucedemo.com`), a public demo e-commerce website with login, product catalogue, shopping cart and checkout, using Selenium WebDriver driven from Python with the pytest framework.

The passing test cases from the companion manual testing project were selected for automation. The suite is organised with a simple Page Object Model: four small classes (LoginPage, InventoryPage, CartPage, CheckoutPage) hold the element locators and user actions of each page, so that tests read like user stories and locator changes are confined to one file. A pytest fixture starts a fresh headless Microsoft Edge browser for every test and closes it afterwards; a hook captures a screenshot whenever a test fails and embeds it in a self-contained HTML report produced by pytest-html. Parametrisation runs the same login and checkout validation logic with several data sets, and a known defect of the `problem_user` account is kept in the suite as an expected failure.

The final suite contains 22 tests across five modules (login, products, cart, checkout, navigation). A full run in headless Edge takes about 105 seconds and the latest execution reports 21 passed and 1 expected failure. The project demonstrates locator strategies, explicit waits, fixtures, data-driven tests, reporting and failure diagnosis, which are the core skills of Selenium-based test automation.

CONTENTS

CHAPTER		TITLE	PAGE NO
1		**Problem Statement Definition**	6
2		**Software Requirements Specification**	6
	2.1	**Introduction**	6
	2.2	**Scope**	6
	2.3	**Requirements**	6
3		**System Implementation**	7
	3.1	**Technologies**	7
	3.2	**Working of Selenium WebDriver**	7
4		**Working**	8
	4.1	**Framework Structure**	8
	4.2	**Automated Test Cases**	9
	4.3	**Execution and Report**	10
5		**Advantages and Limitations**	13
6		**Conclusion**	13

1. Problem Statement Definition

Design and implement an automated functional regression test suite for an e-commerce website (Swag Labs) using Selenium WebDriver, covering login, product listing and sorting, shopping cart, checkout and navigation, with reusable page objects, data-driven test cases, automatic screenshots on failure and an HTML execution report, so that the whole regression run needs no human interaction.

2. Software Requirements Specification

2.1 Introduction

Selenium WebDriver is an open-source tool that controls a real web browser from a program. A test script written in Python tells the browser to open a URL, find elements on the page (by ID, class name, CSS selector or XPath), type text, click buttons and read what is displayed. Assertions then compare what the page shows with what is expected. pytest is a Python testing framework that discovers test functions, provides fixtures for setup and teardown, supports data-driven tests through parametrisation and, with the pytest-html plugin, writes an HTML report. In this project these tools are used to automate the regression test cases of the Swag Labs online store.

2.2 Scope

In scope: functional tests of login (positive and negative), product listing and sorting, cart operations, checkout validation and totals, order completion, logout and protected-page access, in Microsoft Edge (Chrome supported through a command-line option).

Out of scope: visual or pixel comparison, performance measurement, mobile browsers, and tests of the deliberately broken users except one documented expected failure.

2.3 Requirements

Software requirements are:

- Windows 11 (Linux and macOS are also supported)
- Python 3.14
- Selenium 4.51 (Python bindings); Selenium Manager downloads the browser driver automatically
- pytest 9 and pytest-html 4
- Microsoft Edge 142 or Google Chrome

- Visual Studio Code (any editor), Git and GitHub
- Internet connection to `https://www.saucedemo.com`

Hardware: any computer with a 2 GHz dual-core processor and 4 GB RAM.

3. System Implementation

3.1 Technologies

Python is the scripting language used for the tests. It is simple to read, has official Selenium bindings and a very popular test framework (pytest), so the whole suite is short and easy to explain.

Selenium WebDriver provides the browser automation. The script creates a driver object (`webdriver.Edge`), navigates with `driver.get(url)`, locates elements with `find_element(By.ID, "login-button")` and interacts through `send_keys()` and `click()`. Waits (`implicitly_wait`, `WebDriverWait`) make the script wait for the page instead of using fixed sleeps.

pytest discovers every function whose name starts with `test_`, runs the fixture that gives it a browser, executes the assertions and records pass or fail. `@pytest.mark.parametrize` runs one function with several data sets; `@pytest.mark.xfail` marks a test that is expected to fail because of a known defect.

pytest-html writes a single self-contained HTML report with the result and duration of every test; screenshots of failed tests are embedded in it.

Page Object Model is a design pattern in which one class represents one page of the application. The class holds the locators of that page and methods for the user actions. Tests call these methods instead of using raw locators, so if the site changes only the page class has to be updated.

3.2 Working of Selenium WebDriver

Selenium WebDriver works as a client-server system. The Python test is the client. It sends commands (open URL, find element, click, get text) over HTTP to a small server program called the browser driver (`msedgedriver` for Edge, `chromedriver` for Chrome) using the W3C WebDriver protocol. The driver translates each command into native browser actions and sends back the result. Selenium 4 includes Selenium Manager, which downloads the driver matching the installed browser version automatically, so no manual driver setup is needed.

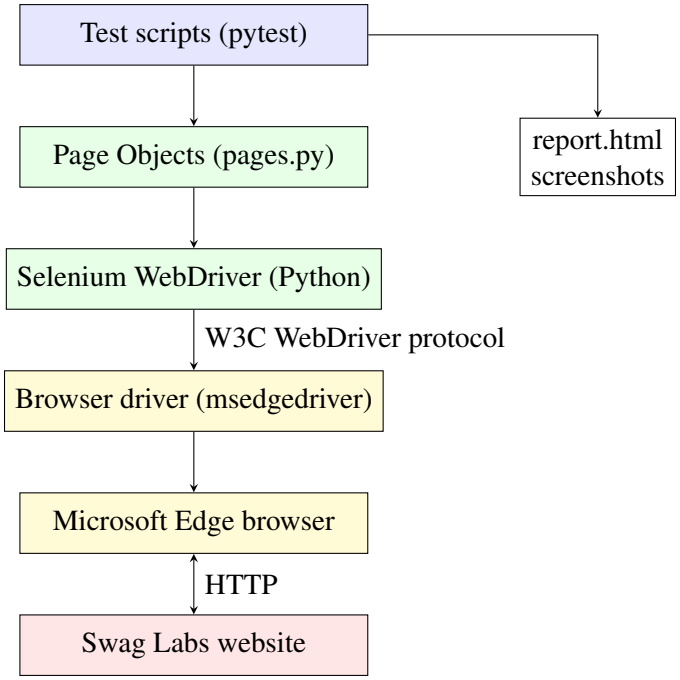


Figure 1: Working of the automation framework

4. Working

4.1 Framework Structure

Repository: `https://github.com/VishwajeetTulse/ecommerce-automation-testing`

```
tests/pages.py           Page Objects: LoginPage, InventoryPage, CartPage,
    CheckoutPage
tests/conftest.py        WebDriver fixture, login fixtures, screenshot-on-failure hook
tests/test_login.py      6 tests      tests/test_products.py    5 tests
tests/test_cart.py       4 tests      tests/test_checkout.py    5 tests
tests/test_navigation.py 2 tests
pytest.ini               default options (verbose, HTML report)
reports/                 report.html, console_output.txt, screenshots/
```

The driver fixture gives every test a fresh browser:

```
@pytest.fixture
def driver(request):
    options = webdriver.EdgeOptions()
    if not request.config.getoption("--headed"):
        options.add_argument("--headless=new")
    options.add_argument("--window-size=1280,900")
    drv = webdriver.Edge(options=options)
    drv.implicitly_wait(5)
    yield drv
```

```
    drv.quit()
```

The LoginPage page object:

```
class LoginPage:
    USERNAME_INPUT = (By.ID, "user-name")
    PASSWORD_INPUT = (By.ID, "password")
    LOGIN_BTN = (By.ID, "login-button")
    ERROR = (By.CSS_SELECTOR, "h3[data-test='error']")

    def login(self, username, password=DEFAULT_PASSWORD):
        self.driver.find_element(*self.USERNAME_INPUT).send_keys(username)
        self.driver.find_element(*self.PASSWORD_INPUT).send_keys(password)
        self.driver.find_element(*self.LOGIN_BTN).click()
        WebDriverWait(self.driver, 10).until(
            lambda d: "inventory.html" in d.current_url or d.find_elements(*self.ERROR
    ))
        return InventoryPage(self.driver)
```

4.2 Automated Test Cases

A data-driven negative login test and the checkout totals test:

```
@pytest.mark.parametrize("username,password,expected", [
    ("", "", "Username is required"),
    ("standard_user", "", "Password is required"),
    ("standard_user", "wrong", "do not match any user"),
    ("locked_out_user", "secret_sauce", "locked out"),
])
def test_invalid_login_shows_error(login_page, username, password, expected):
    login_page.login(username, password)
    assert expected in login_page.error_message()

def test_overview_totals_are_correct(checkout):
    checkout.fill_information("Vishwajeet", "Tulse", "411041")
    subtotal, tax, total = checkout.subtotal(), checkout.tax(), checkout.total()
    assert subtotal == pytest.approx(39.98)
    assert tax == pytest.approx(round(subtotal * 0.08, 2))
    assert total == pytest.approx(subtotal + tax)
```

Test	What is verified	Result
test_valid_login	standard_user reaches the Products page	Pass
test_invalid_login_shows_error (4 data sets)	empty, no password, wrong password, locked-out user show the correct error	Pass
test_password_is_masked	password input type is “password”	Pass
test_six_products_listed	six product names on the Products page	Pass
test_sort_price_low_to_high / high_to_low	prices ordered after sorting	Pass
test_sort_name_z_to_a	names in reverse alphabetical order	Pass
test_sort_price_problem_user	sorting for problem_user (known defect)	XFAIL
test_add_two_items_updates_badge	cart badge shows 2	Pass
test_cart_page_lists_added_items	both items listed on the cart page	Pass
test_remove_item_from_cart_page	one item left, badge 1	Pass
test_remove_from_products_page	badge disappears	Pass
test_missing_information_shows_error (3 data sets)	first name, last name, postal code required	Pass
test_overview_totals_are_correct	subtotal 39.98, tax 8%, total = sum	Pass
test_complete_order	“Thank you for your order!” page	Pass
test_logout_returns_to_login_page	login page after logout	Pass
test_inventory_page_requires_login	direct URL shows the access error	Pass

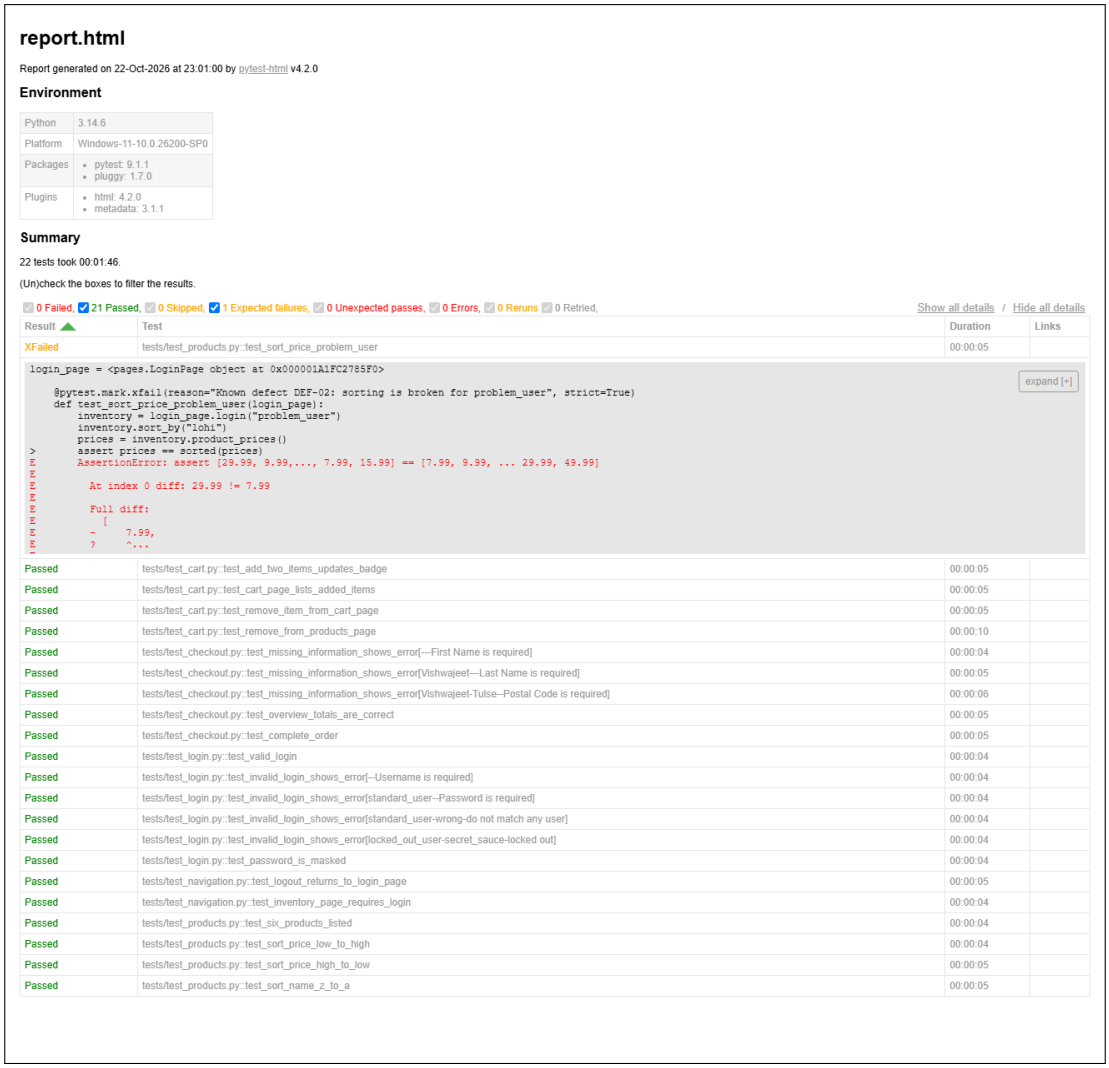
4.3 Execution and Report

```
pip install -r requirements.txt
python -m pytest                      # headless Edge, writes reports/report.html
```

```
python -m pytest --headed             # watch the browser
python -m pytest --browser chrome     # use Chrome
```

Console output of the final run:

```
tests/test_cart.py::test_add_two_items_updates_badge PASSED
tests/test_cart.py::test_cart_page_lists_added_items PASSED
tests/test_cart.py::test_remove_item_from_cart_page PASSED
tests/test_cart.py::test_remove_from_products_page PASSED
tests/test_checkout.py::test_missing_information_shows_error[---First Name is required
    ] PASSED
tests/test_checkout.py::test_missing_information_shows_error[Vishwajeet---Last Name is
     required] PASSED
tests/test_checkout.py::test_missing_information_shows_error[Vishwajeet-Tulse--Postal
    Code is required] PASSED
tests/test_checkout.py::test_overview_totals_are_correct PASSED
tests/test_checkout.py::test_complete_order PASSED
tests/test_login.py::test_valid_login PASSED
tests/test_login.py::test_invalid_login_shows_error[--Username is required] PASSED
tests/test_login.py::test_invalid_login_shows_error[standard_user--Password is
    required] PASSED
tests/test_login.py::test_invalid_login_shows_error[standard_user-wrong-do not match
    any user] PASSED
tests/test_login.py::test_invalid_login_shows_error[locked_out_user-secret_sauce-
    locked out] PASSED
tests/test_login.py::test_password_is_masked PASSED
tests/test_navigation.py::test_logout_returns_to_login_page PASSED
tests/test_navigation.py::test_inventory_page_requires_login PASSED
tests/test_products.py::test_six_products_listed PASSED
tests/test_products.py::test_sort_price_low_to_high PASSED
tests/test_products.py::test_sort_price_high_to_low PASSED
tests/test_products.py::test_sort_name_z_to_a PASSED
tests/test_products.py::test_sort_price_problem_user XFAIL (Known defect DEF-02)
================== 21 passed, 1 xfailed in 105.64s (0:01:45) ==================
```



report.html

Report generated on 22-Oct-2026 at 23:01:00 by pytest-html v4.2.0

Environment

Python	3.14.6
Platform	Windows-11-10.0.26200-SP0
Packages	• pytest: 9.1.1 • pluggy: 1.7.0
Plugins	• html: 4.2.0 • metadata: 3.1.1

Summary

22 tests took 00:01:46.

(Un)check the boxes to filter the results.

0 Failed, 21 Passed, 0 Skipped, 1 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns 0 Retried, Show all details / Hide all details

Result	Test	Duration	Links
XFailed	tests/test_products.py::test_sort_price_problem_user	00:00:05	

```
login_page = <pages.LoginPage object at 0x000001A1FC2785F0>

    @pytest.mark.xfail(reason="Known defect DEF-02: sorting is broken for problem_user", strict=True)
    def test_sort_price_problem_user(login_page):
        inventory = login_page.login("problem_user")
        inventory.sort_by("lohi")
        prices = inventory.product_prices()
>       assert prices == sorted(prices)
E       AssertionError: assert [29.99, 9.99,..., 7.99, 15.99] == [7.99, 9.99, ... 29.99, 49.99]
E
E         At index 0 diff: 29.99 != 7.99
E
E         Full diff:
E           [
E         -     7.99,
E         ?     ^...
```

expand [+]

Passed	tests/test_cart.py::test_add_two_items_updates_badge	00:00:05
Passed	tests/test_cart.py::test_cart_page_lists_added_items	00:00:05
Passed	tests/test_cart.py::test_remove_item_from_cart_page	00:00:05
Passed	tests/test_cart.py::test_remove_from_products_page	00:00:10
Passed	tests/test_checkout.py::test_missing_information_shows_error[---First Name is required]	00:00:04
Passed	tests/test_checkout.py::test_missing_information_shows_error[Vishwajeet---Last Name is required]	00:00:05
Passed	tests/test_checkout.py::test_missing_information_shows_error[Vishwajeet-Tulse--Postal Code is required]	00:00:06
Passed	tests/test_checkout.py::test_overview_totals_are_correct	00:00:05
Passed	tests/test_checkout.py::test_complete_order	00:00:05
Passed	tests/test_login.py::test_valid_login	00:00:04
Passed	tests/test_login.py::test_invalid_login_shows_error[--Username is required]	00:00:04
Passed	tests/test_login.py::test_invalid_login_shows_error[standard_user--Password is required]	00:00:04
Passed	tests/test_login.py::test_invalid_login_shows_error[standard_user-wrong-do not match any user]	00:00:04
Passed	tests/test_login.py::test_invalid_login_shows_error[locked_out_user-secret_sauce-locked out]	00:00:04
Passed	tests/test_login.py::test_password_is_masked	00:00:04
Passed	tests/test_navigation.py::test_logout_returns_to_login_page	00:00:05
Passed	tests/test_navigation.py::test_inventory_page_requires_login	00:00:04
Passed	tests/test_products.py::test_six_products_listed	00:00:04
Passed	tests/test_products.py::test_sort_price_low_to_high	00:00:04
Passed	tests/test_products.py::test_sort_price_high_to_low	00:00:05
Passed	tests/test_products.py::test_sort_name_z_to_a	00:00:05

Figure 2: pytest-html report of the final run

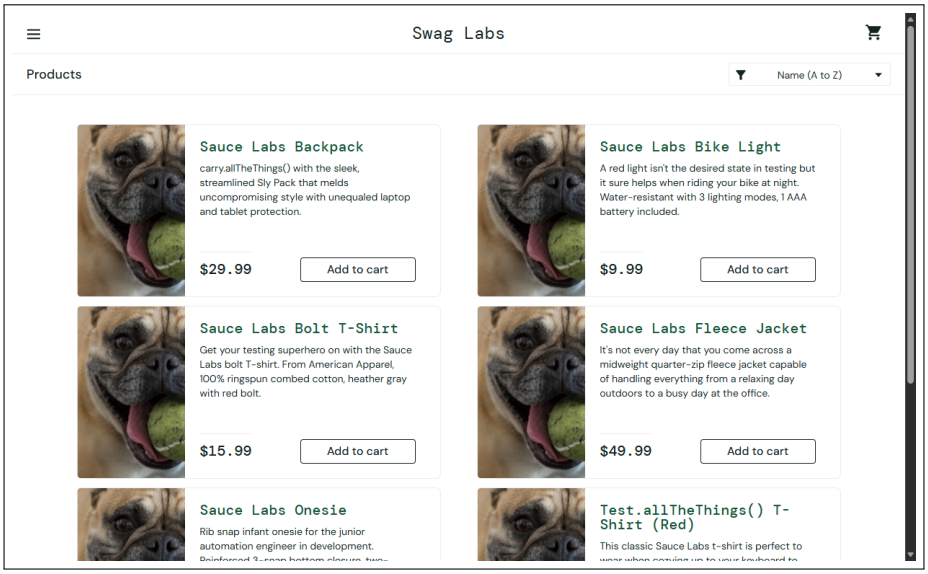


Figure 3: Screenshot captured automatically when a test fails (prices not sorted for problem_user)

5. Advantages and Limitations

Advantages

- The complete regression suite runs unattended in about two minutes instead of about two hours by hand.
- Tests are repeatable and give the same result every time; no human error.
- Page objects keep locators in one file, so maintenance after a site change is small.
- Data-driven tests cover several inputs with one function.
- Failures are easy to diagnose because a screenshot and an HTML report are produced automatically.
- The suite can be plugged into a CI pipeline such as GitHub Actions.

Limitations

- Writing the scripts needs programming effort before the first benefit is seen.
- Automated tests only check what they are told to check; they do not notice new visual or usability problems.
- Scripts break when the application's DOM changes and must be maintained.
- Timing issues (page not yet loaded) can cause flaky results if waits are not used correctly.
- Only Edge and Chrome on desktop are covered.

6. Conclusion

This project automated the regression test cases of an e-commerce website with Selenium WebDriver, Python and pytest. A small Page Object Model keeps locators out of the tests, fixtures give every test an isolated browser, parametrisation covers several data sets with one function, and an HTML report with automatic screenshots makes failures easy to diagnose. The final suite of 22 tests runs unattended in under two minutes and passes except for one documented defect kept as an expected failure. During development the screenshot-on-failure feature immediately revealed a scripting mistake in the login page object, showing the value of good diagnostics. In future the suite can run in a continuous-integration pipeline on every commit, be extended to Selenium Grid for cross-browser runs, and be linked to a test-management tool for traceability to the manual test cases.